

Unit 4 : Recurrent Neural Network for NLP

Mr. himanshu tiwari

Deep Learning with NLP(303105479)

Artificial Intelligence and Data Science Department

Content

1. What is RNN and Recurrence
2. Difference between Feedforward and Recurrent Neural Networks
3. Basics of Recurrent Neural Networks
4. Natural Language Processing & Recurrent Neural Networks
5. RNN's Mechanism and Training RNN
6. Meta meaning of Hidden State of RNN
7. Tuning RNN
8. Long Short-Term Memory Networks
9. Sequence to Sequence Models
10. Advance Sequence to Sequence Models
11. References

What is RNN and Recurrence?

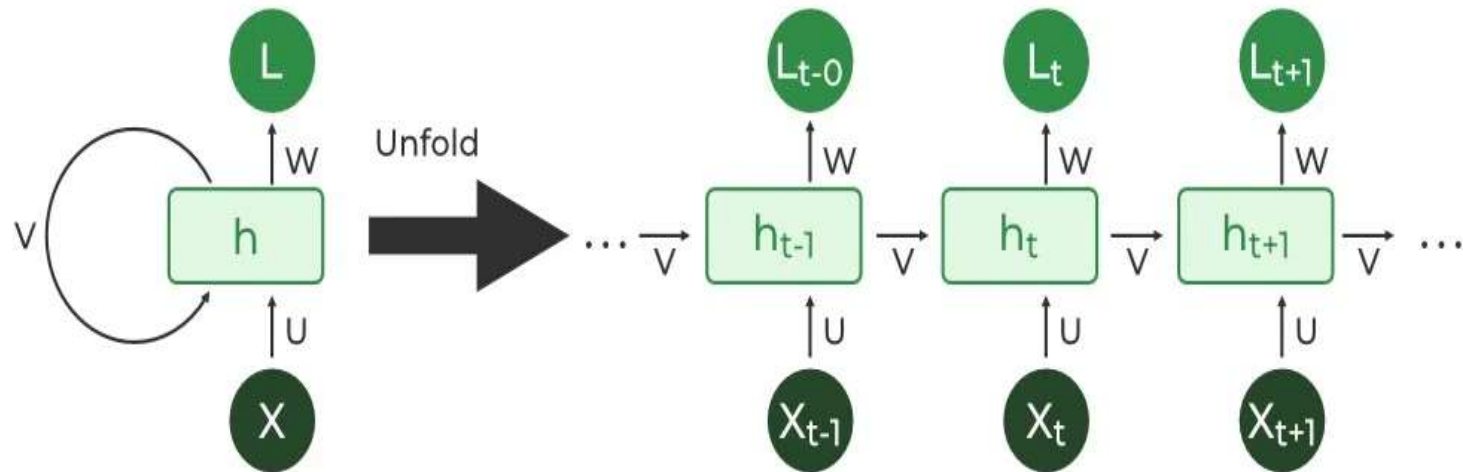
Introduction:

- Recurrent Neural Network(RNN) is a type of Neural Network where the output from the previous step is fed as input to the current step.
- In traditional neural networks, all the inputs and outputs are independent of each other, but in cases when it is required to predict the next word of a sentence, the previous words are required and hence there is a need to remember the previous words.
- Thus RNN came into existence, which solved this issue with the help of a Hidden Layer. The main and most important feature of RNN is its Hidden state, which remembers some information about a sequence.

What is RNN and Recurrence?

- The state is also referred to as **Memory State** since it remembers the previous input to the network.
- It uses the same parameters for each input as it performs the same task on all the inputs or hidden layers to produce the output.
- This reduces the complexity of parameters, unlike other neural networks.
- The term "recurrence" in RNN refers to the network's ability to maintain a hidden state that captures information from previous time steps.
- In essence, recurrence refers to the feedback loop that allows RNNs to process sequences.

What is RNN and Recurrence?



Difference between Feedforward and Recurrent Neural Networks

Comparison Attribute	Feed-forward Neural Networks	Recurrent Neural Networks
Signal flow direction	Forward only	Bidirectional
Delay introduced	No	Yes
Complexity	Low	High
Neuron independence in the same layer	Yes	No
Speed	High	slow
Commonly used for	Pattern recognition, speech recognition, and character recognition	Language translation, speech-to-text conversion, and robotic control
Data	used for tasks with fixed-size input data such as image classification	well-suited for tasks involving sequential data such as time series forecasting and NLP

Basics of Recurrent Neural Networks

How RNN works

- The Recurrent Neural Network consists of multiple fixed activation function units, one for each time step.
- Each unit has an internal state which is called the hidden state of the unit. This hidden state signifies the past knowledge that the network currently holds at a given time step.
- This hidden state is updated at every time step to signify the change in the knowledge of the network about the past.
- The hidden state is updated using the following recurrence relation:-
- The formula for calculating the current state:

$$h_t = f(h_{t-1}, x_t)$$

Natural Language Processing & Recurrent Neural Networks

NLP Applications:

In NLP, RNNs are widely used for various applications, including:

1. Language Modeling: Predicting the next word in a sentence.
2. Text Generation: Generating coherent text, such as in chatbots.
3. Sentiment Analysis: Determining the sentiment (positive/negative) of text.
4. Machine Translation: Translating text from one language to another.
5. Summarization: Creating concise summaries of longer text
6. Word Embeddings: RNNs in NLP often use word embeddings, which are pre-trained models that map words into high dimensional vectors. Word embeddings capture semantic relationships between words, improving the model's ability to understand and generate human language.

Natural Language Processing & Recurrent Neural Networks

7. Sequence-to-Sequence Models: RNNs play a central role in sequence-to-sequence models.

- These models use an RNN encoder to process input sequences and an RNN decoder to generate output sequences.
- Sequence-to-sequence models are essential in machine translation, where they convert sentences from one language to another.

Challenges: Challenges in NLP with RNNs include handling variable-length sentences, addressing the vanishing gradient problem (The gradients contain information utilized in the RNN parameter update, and as the gradient shrinks, the parameter updates become minor, implying that no meaningful learning occurs) and dealing with large-scale models.

Specialized RNN variants like Long Short-Term Memory (LSTM) and Gated Recurrent Unit (GRU) have been developed to mitigate some of these challenges.

Natural Language Processing & Recurrent Neural Networks

RNN's Mechanism and Training RNN

Types Of RNN

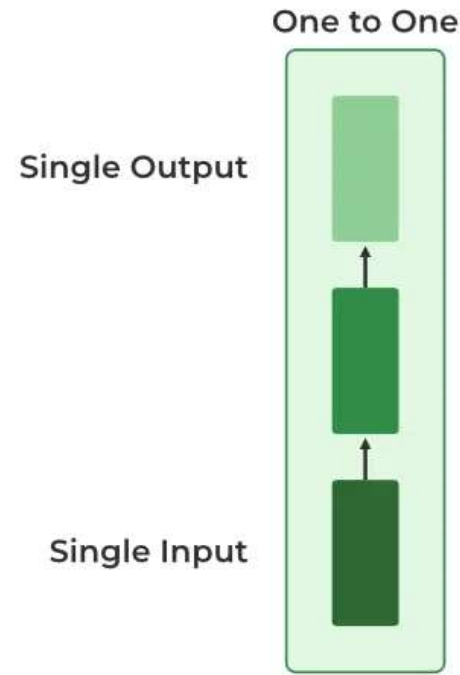
There are four types of RNNs based on the number of inputs and outputs in the network.

1. One to One
2. One to Many
3. Many to One
4. Many to Many

RNN's Mechanism and Training RNN

One to One

This type of RNN behaves the same as any simple Neural network it is also known as Vanilla Neural Network. In this Neural network, there is only one input and one output.



RNN's Mechanism and Training RNN

Many to One

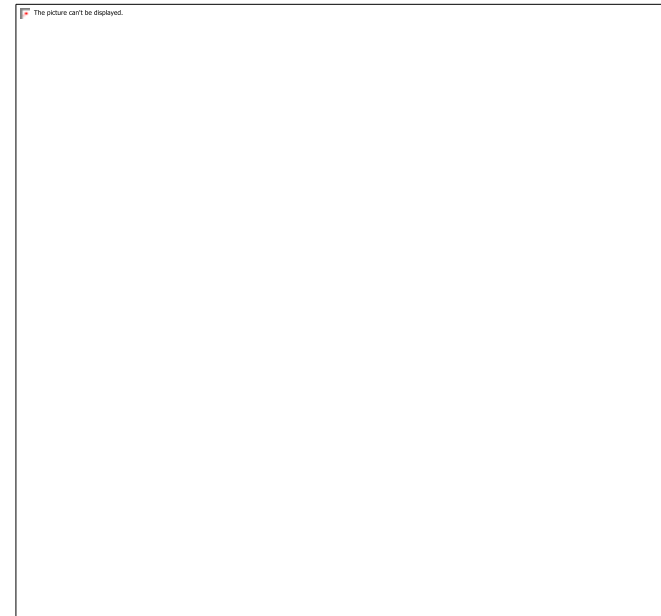
In this type of network, Many inputs are fed to the network at several states of the network generating only one output. This type of network is used in the problems like sentimental analysis. Where we give multiple words as input and predict only the sentiment of the sentence as output.

The picture can't be displayed.

RNN's Mechanism and Training RNN

One To Many

In this type of RNN, there is one input and many outputs associated with it. One of the most used examples of this network is Image captioning where given an image we predict a sentence having Multiple words.



RNN's Mechanism and Training RNN

Many to Many

In this type of neural network, there are multiple inputs and multiple outputs corresponding to a problem. One Example of this Problem will be language translation. In language translation, we provide multiple words from one language as input and predict multiple words from the second language as output.

The picture can't be displayed.

RNN's Mechanism and Training RNN

Training through RNN

- A single-time step of the input is provided to the network.
- Then calculate its current state using a set of current input and the previous state.
- The current h_t becomes h_{t-1} for the next time step.
- One can go as many time steps according to the problem and join the information from all the previous states.
- Once all the time steps are completed the final current state is used to calculate the output.
- The output is then compared to the actual output i.e the target output and the error is generated.
- The error is then back-propagated to the network to update the weights and hence the network (RNN) is trained using Backpropagation through time.

RNN's Mechanism and Training RNN

Advantages of Recurrent Neural Network

An RNN remembers each and every piece of information through time. It is useful in time series prediction only because of the feature to remember previous inputs as well. This is called Long Short Term Memory.

Recurrent neural networks are even used with convolutional layers to extend the effective pixel neighborhood.

Disadvantages of Recurrent Neural Network

- „Gradient vanishing and exploding problems.

- Training an RNN is a very difficult task.

- It cannot process very long sequences if using tanh or relu as an activation function.

RNN's Mechanism and Training RNN

Applications of Recurrent Neural Network

1. Language Modelling and Generating Text
2. Speech Recognition
3. Machine Translation
4. Image Recognition, Face detection
5. Time series Forecasting

.

Problem in Short term Memory

- Recurrent Neural Networks suffer from short-term memory. If a sequence is long enough, they will have a hard time carrying information from earlier time steps to later ones.
- So if you are trying to process a paragraph of text to do predictions, RNN's may leave out important information from the beginning.
- During back propagation, recurrent neural networks suffer from the vanishing gradient problem. Gradients are values used to update a neural networks weights.
- The vanishing gradient problem is when the gradient shrinks as it back propagates through time. If a gradient value becomes extremely small, it doesn't contribute too much learning.

new weight = weight - learning rate*gradient

$$\boxed{2.0999} = \boxed{2.1}$$

Not much of a difference

-

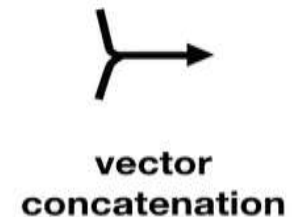
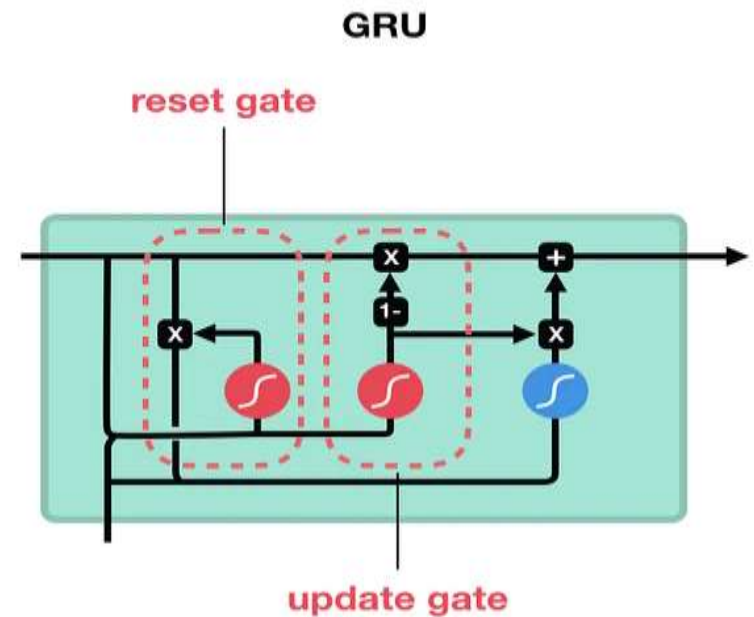
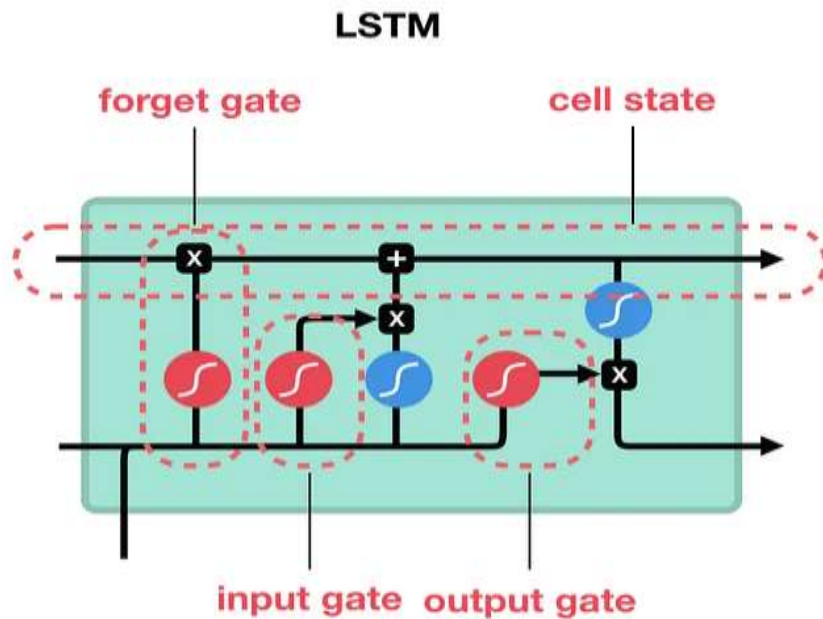
$$\boxed{0.001}$$

update value

Problem in Short term Memory

- So in recurrent neural networks, layers that get a small gradient update stops learning. Those are usually the earlier layers.
- So because these layers don't learn, RNN's can forget what it seen in longer sequences, thus having a short-term memory.
- LSTM 's and GRU's were created as the solution to short-term memory. They have internal mechanisms called gates that can regulate the flow of information.
- What is the difference between LSTM and Gated Recurrent Unit (GRU)?
- LSTM and GRU are both variants of RNN that are used to resolve the vanishing gradient issue of the RNN, but they have some differences, which are:
 - 1. LSTM uses three gates to compute the input of sequence data where, whereas GRU uses only two gates.
 - 2. GRUs are generally simpler and faster than LSTM.
 - 3. LSTMs are preferred for large datasets, whereas for small datasets GRUs are preferred.

Problem in Short term Memory



Long Short-Term Memory Networks

- Long Short-Term Memory Networks is a deep learning, sequential neural network that allows information to persist. It is a special type of Recurrent Neural Network which is capable of handling the vanishing gradient problem faced by RNN.
- LSTM was designed by Hochreiter and Schmidhuber that resolves the problem caused by traditional rnns and machine learning algorithms. LSTM can be implemented in Python using the Keras library.
- Let's say while watching a video, you remember the previous scene, or while reading a book, you know what happened in the earlier chapter. RNNs work similarly; they remember the previous information and use it for processing the current input.
- The shortcoming of RNN is they cannot remember long-term dependencies due to vanishing gradient. LSTMs are explicitly designed to avoid long-term dependency problems.

Long Short-Term Memory Networks

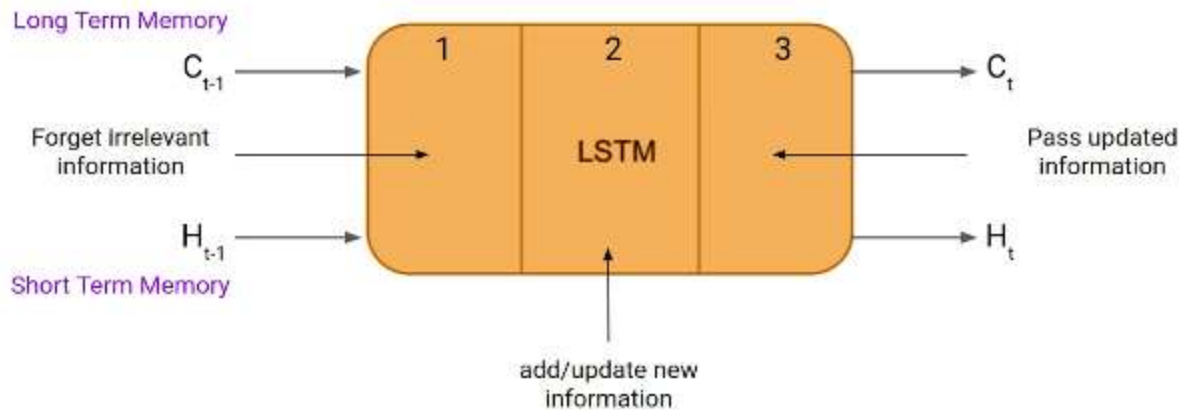
LSTM (Long Short-Term Memory) is a recurrent neural network (RNN) architecture widely used in Deep Learning. It excels at capturing long-term dependencies, making it ideal for sequence prediction tasks.

Unlike traditional neural networks, LSTM incorporates feedback connections, allowing it to process entire sequences of data, not just individual data points. This makes it highly effective in understanding and predicting patterns in sequential data like time series, text, and speech.

LSTM has become a powerful tool in artificial intelligence and deep learning, enabling breakthroughs in various fields by uncovering valuable insights from sequential data.

Long Short-Term Memory Networks

- Here the hidden state is known as Short term memory, and the cell state is known as Long term memory. Refer to the following image.



It is interesting to note that the cell state carries the information along with all the timestamps.

Long Short-Term Memory Networks



LSTM

Bob is a nice person. Dan on the other hand is evil.

Long Short-Term Memory Networks

Forget Gate:

- $f_t = \sigma(x_t * U_f + H_{t-1} * W_f)$

Let's try to understand the equation, here

- x_t : input to the current timestamp.
- U_f : weight associated with the input
- H_{t-1} : The hidden state of the previous timestamp
- W_f : It is the weight matrix associated with the hidden state

Later, a sigmoid function is applied to it. That will make f_t a number between 0 and 1. This f_t is later multiplied with the cell state of the previous timestamp, as shown below.

$$C_{t-1} * f_t = 0 \quad \dots \text{if } f_t = 0 \text{ (forget everything)}$$

$$C_{t-1} * f_t = C_{t-1} \quad \dots \text{if } f_t = 1 \text{ (forget nothing)}$$

Long Short-Term Memory Networks

- **Input Gate**
- Let's take another example.
- “Bob knows swimming. He told me over the phone that he had served the navy for four long years.”
- So, in both these sentences, we are talking about Bob. However, both give different kinds of information about Bob. In the first sentence, we get the information that he knows swimming. Whereas the second sentence tells, he uses the phone and served in the navy for four years.
- Now just think about it, based on the context given in the first sentence, which information in the second sentence is critical? First, he used the phone to tell, or he served in the navy. In this context, it doesn't matter whether he used the phone or any other medium of communication to pass on the information. The fact that he was in the navy is important information, and this is something we want our model to remember for future computation. This is the task of the Input gate.

Long Short-Term Memory Networks

- The input gate is used to quantify the importance of the new information carried by the input. Here is the equation of the input gate

Input Gate:

- $i_t = \sigma(x_t * U_i + H_{t-1} * W_i)$

Here,

- X_t : Input at the current timestamp t
- U_i : weight matrix of input
- H_{t-1} : A hidden state at the previous timestamp
- W_i : Weight matrix of input associated with hidden state

Again we have applied the sigmoid function over it. As a result, the value of I at timestamp t will be between 0 and 1.

New Information

- $N_t = \tanh(x_t * U_c + H_{t-1} * W_c)$ (new information)

Long Short-Term Memory Networks

- Now the new information that needed to be passed to the cell state is a function of a hidden state at the previous timestamp $t-1$ and input x at timestamp t . The activation function here is $\tanh$. Due to the $\tanh$ function, the value of new information will be between -1 and 1 . If the value of N_t is negative, the information is subtracted from the cell state, and if the value is positive, the information is added to the cell state at the current timestamp.
- However, the N_t won't be added directly to the cell state. Here comes the updated equation:

$$C_t = f_t * C_{t-1} + i_t * N_t \text{ (updating cell state)}$$

- Here, C_{t-1} is the cell state at the current timestamp, and the others are the values we have calculated previously.

Long Short-Term Memory Networks

Output Gate

Now consider this sentence.

“Bob single-handedly fought the enemy and died for his country. For his contributions, brave_____.”

During this task, we have to complete the second sentence. Now, the minute we see the word brave, we know that we are talking about a person. In the sentence, only Bob is brave, we can not say the enemy is brave, or the country is brave. So based on the current expectation, we have to give a relevant word to fill in the blank. That word is our output, and this is the function of our Output gate.

Here is the equation of the Output gate, which is pretty similar to the two previous gates.

Output Gate:

- $$o_t = \sigma(x_t * U_o + H_{t-1} * W_o)$$

Its value will also lie between 0 and 1 because of this sigmoid function.

Long Short-Term Memory Networks

Now to calculate the current hidden state, we will use O_t and $\tanh$ of the updated cell state. As shown below

$$H_t = o_t * \tanh(C_t)$$

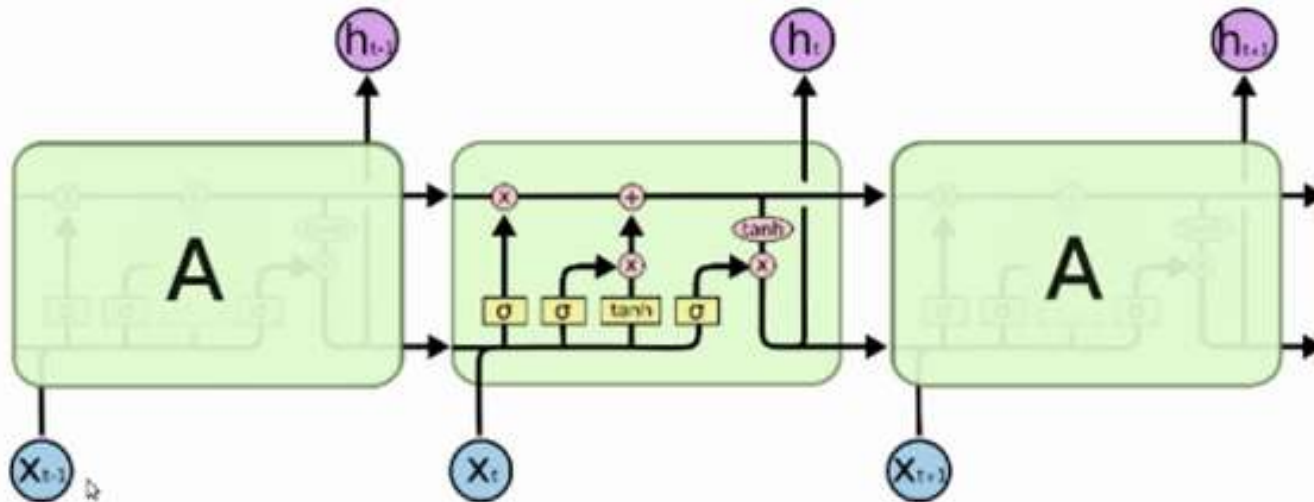
It turns out that the hidden state is a function of Long term memory (C_t) and the current output. If you need to take the output of the current timestamp, just apply the SoftMax activation on hidden state H_t .

$$\text{Output} = \text{Softmax}(H_t)$$

Here the token with the maximum score in the output is the prediction.

Long Short-Term Memory Networks

This is the More intuitive diagram of the LSTM network.

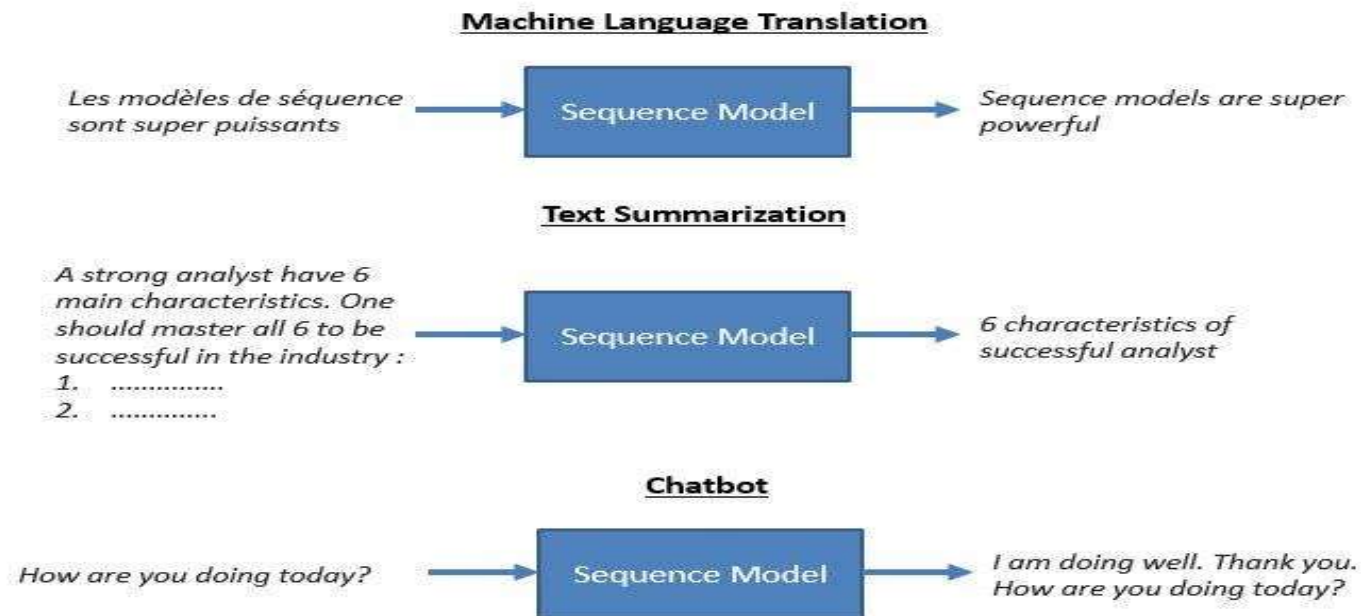


LTSM vs RNN

Aspect	LSTM (Long Short-Term Memory)	RNN (Recurrent Neural Network)
Architecture	A type of RNN with additional memory cells	A basic type of RNN
Memory Retention	Handles long-term dependencies and prevents vanishing gradient problem	Struggles with long-term dependencies and vanishing gradient problem
Cell Structure	Complex cell structure with input, output, and forget gates	Simple cell structure with only hidden state
Handling Sequences	Suitable for processing sequential data	Also designed for sequential data, but limited memory
Training Efficiency	Slower training process due to increased complexity	Faster training process due to simpler architecture
Performance on Long Sequences	Performs better on long sequences	Struggles to retain information on long sequences
Usage	Best suited for tasks requiring long-term memory, such as language translation and sentiment analysis	Appropriate for simple sequential tasks, such as time series forecasting
Vanishing Gradient Problem	Addresses the vanishing gradient problem	Prone to the vanishing gradient problem

Sequence to Sequence Models

Sequence to Sequence (often abbreviated to seq2seq) models is a special class of Recurrent Neural Network architectures that we typically use (but not restricted) to solve complex Language problems like Machine Translation, Question Answering, creating Chatbots, Text Summarization, etc.



Sequence to Sequence Models

Use Cases of the Sequence to Sequence Models

Sequence to sequence models lies behind numerous systems that you face on a daily basis.

For instance, seq2seq model powers applications like Google Translate, voice-enabled devices, and online chatbots.

The following are some of the applications:

- *Machine translation* — a 2016 [paper](#) from Google shows how the seq2seq model's translation quality “approaches or surpasses all currently published results”.
- *Speech recognition* — another Google [paper](#) that compares the existing seq2seq models on the speech recognition task.
- These are only some applications where seq2seq is seen as the best solution. This model can be used as a solution to any sequence-based problem, especially ones where the inputs and outputs have different sizes and categories.

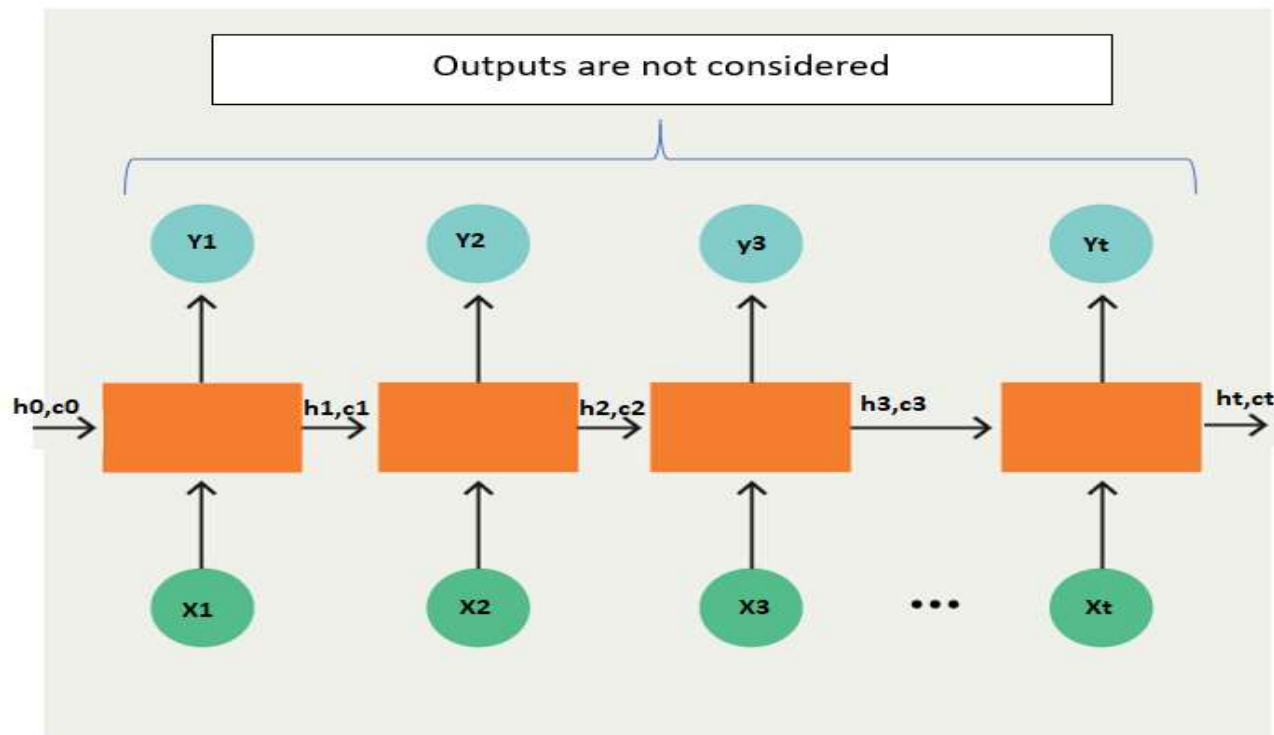
Sequence to Sequence Models

- **Encoder-Decoder Architecture:**
- The most common architecture used to build Seq2Seq models is Encoder-Decoder architecture.
- **Encoder :**
- Both encoder and the decoder are LSTM models (or sometimes GRU models)
- Encoder reads the input sequence and summarizes the information in something called the internal state vectors or context vector (in case of LSTM these are called the hidden state and cell state vectors).
- We discard the outputs of the encoder and only preserve the internal states. This context vector aims to encapsulate the information for all input elements in order to help the decoder make accurate predictions.

Sequence to Sequence Models

The hidden states h_i are computed using the formula:

$$h_t = f(W^{(hh)}h_{t-1} + W^{(hx)}x_t)$$



Sequence to Sequence Models

The LSTM reads the data, one sequence after the other. Thus if the input is a sequence of length 't', we say that LSTM reads it in 't' time steps.

1. X_i = Input sequence at time step i.
2. h_i and c_i = LSTM maintains two states ('h' for hidden state and 'c' for cell state) at each time step. Combined together these are internal state of the LSTM at time step i.
3. Y_i = Output sequence at time step i. Y_i is actually a probability distribution over the entire vocabulary which is generated by using a softmax activation. Thus each Y_i is a vector of size "vocab_size" representing a probability distribution.

Sequence to Sequence Models

Decoder :

- The decoder is an LSTM whose initial states are initialized to the final states of the Encoder LSTM, i.e. the context vector of the encoder's final cell is input to the first cell of the decoder network. Using these initial states, the decoder starts generating the output sequence, and these outputs are also taken into consideration for future outputs.
- A stack of several LSTM units where each predicts an output y_t at a time step t .
- Each recurrent unit accepts a hidden state from the previous unit and produces an output as well as its own hidden state.
- Any hidden state h_t is computed using the formula:

$$h_t = f(W^{(hh)} h_{t-1})$$

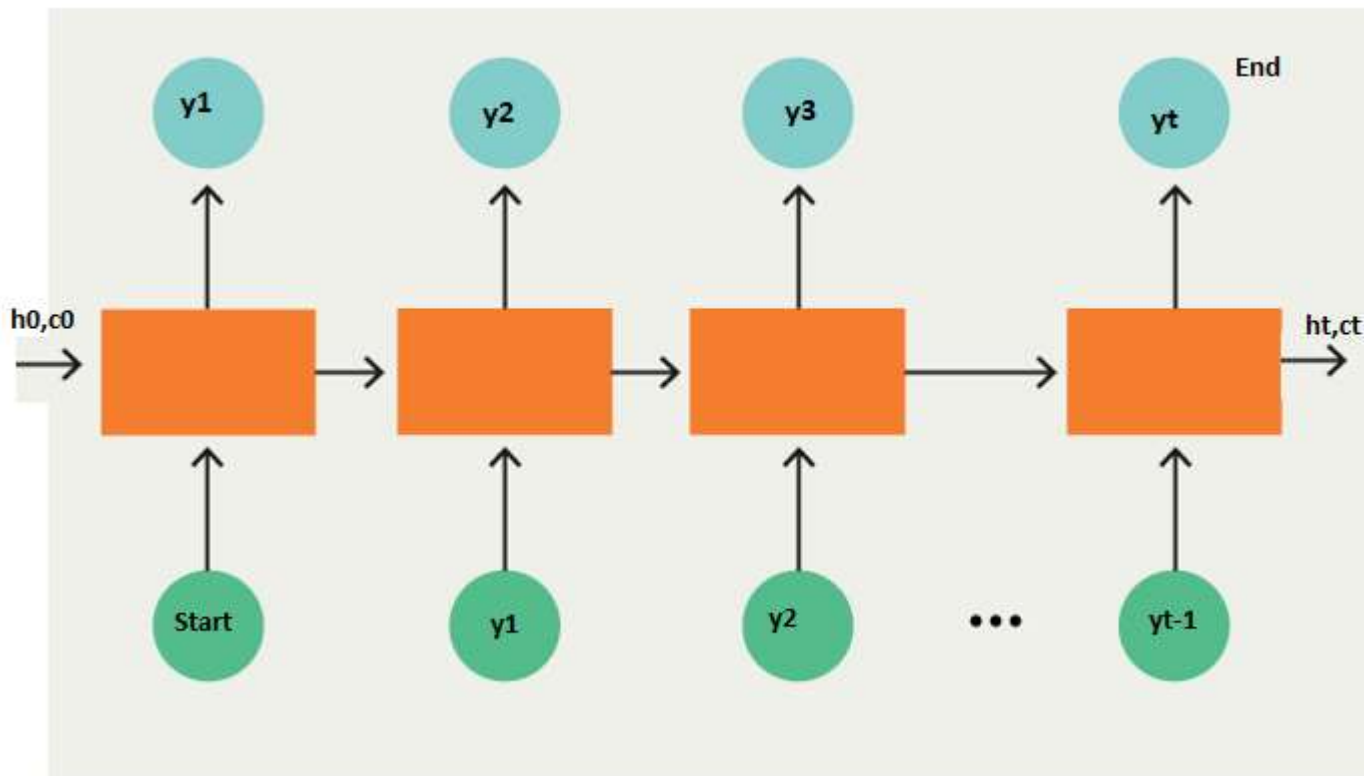
Sequence to Sequence Models

The output y_t at time step t is computed using the formula:

$$y_t = \text{softmax}(W^S h_t)$$

We calculate the outputs using the hidden state at the current time step together with the respective weight $W(S)$. [Softmax](#) is used to create a probability vector which will help us determine the final output (e.g. word in the question-answering problem).

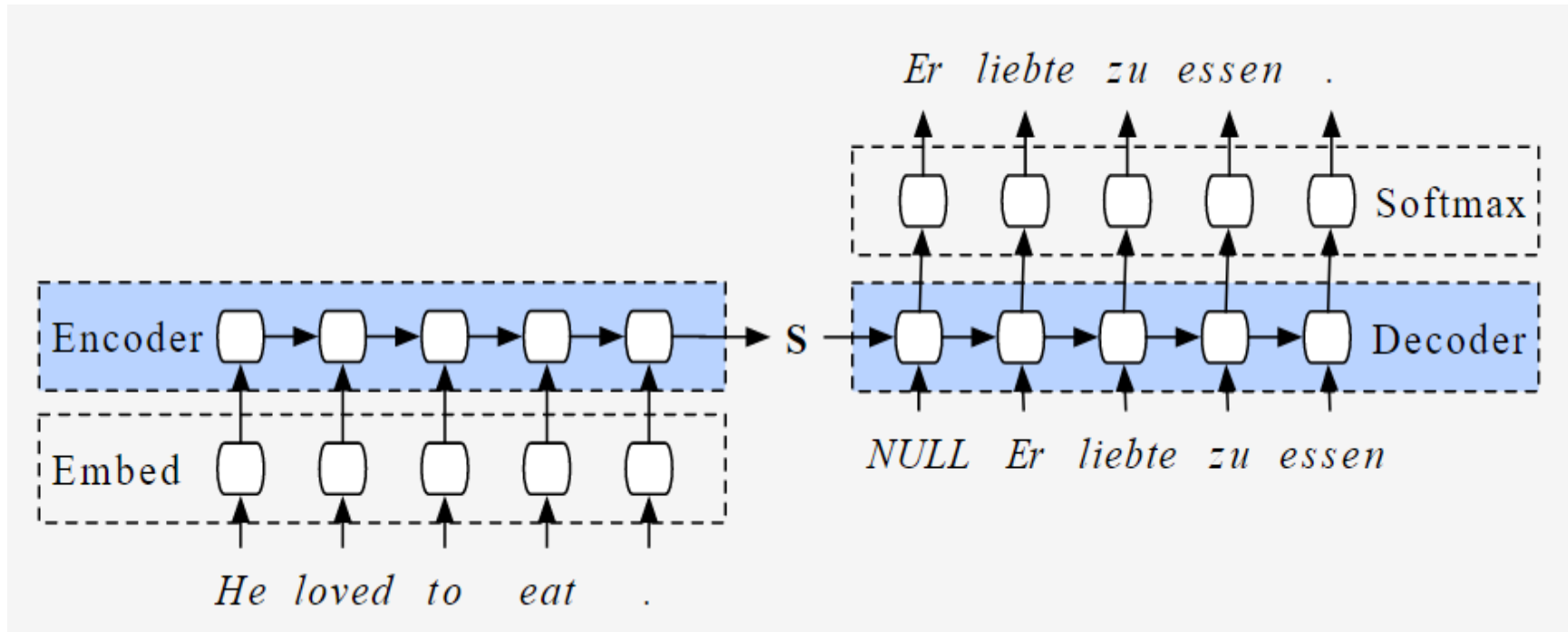
Sequence to Sequence Models



Sequence to Sequence Models

- We will add two tokens in the output sequence as follows:
- Example:
- “**START_** John is hard working **_END**”.
- The most important point is that the initial states (h_0 , c_0) of the decoder are set to the final states of the encoder. This intuitively means that the decoder is trained to start generating the output sequence depending on the information encoded by the encoder.
- Finally, the loss is calculated on the predicted outputs from each time step and the errors are backpropagated through time in order to update the parameters of the network. Training the network over a longer period with a sufficiently large amount of data results in pretty good predictions.

Sequence to Sequence Models



Sequence to Sequence Models

- During inference, we generate one word at a time.
- The initial states of the decoder are set to the final states of the encoder.
- The initial input to the decoder is always the START token.
- At each time step, we preserve the states of the decoder and set them as initial states for the next time step.
- At each time step, the predicted output is fed as input in the next time step.
- We break the loop when the decoder predicts the END token.

Sequence to Sequence Models

Drawbacks of Encoder-Decoder Models :

There are two primary drawbacks to this architecture, both related to length.

1. Firstly, as with humans, this architecture has very limited memory. That final hidden state of the LSTM, which we call **S or W**, is where you're trying to cram the entirety of the sentence you have to translate. **S or W** is usually only a few hundred units (read: floating-point numbers) long — the more you try to force into this fixed dimensionality vector, the lossier the neural network is forced to be. Thinking of neural networks in terms of the “lossy compression” they're required to perform is sometimes quite useful.

2. Second, as a general rule of thumb, the deeper a neural network is, the harder it is to train. For recurrent neural networks, the longer the sequence is, the deeper the neural network is along the time dimension. This results in vanishing gradients, where the gradient signal from the objective that the recurrent neural network learns from disappears as it travels backward. Even with RNNs specifically made to help prevent vanishing gradients, such as the LSTM, this is still a fundamental problem.

Advance Sequence to Sequence Models

Sequence-to-Sequence (Seq2Seq) models are deep learning architectures designed to transform one sequence into another sequence. These models are widely used in Natural Language Processing (NLP) tasks such as:

Machine Translation

Text Summarization

Speech Recognition

Chatbots

Question Answering

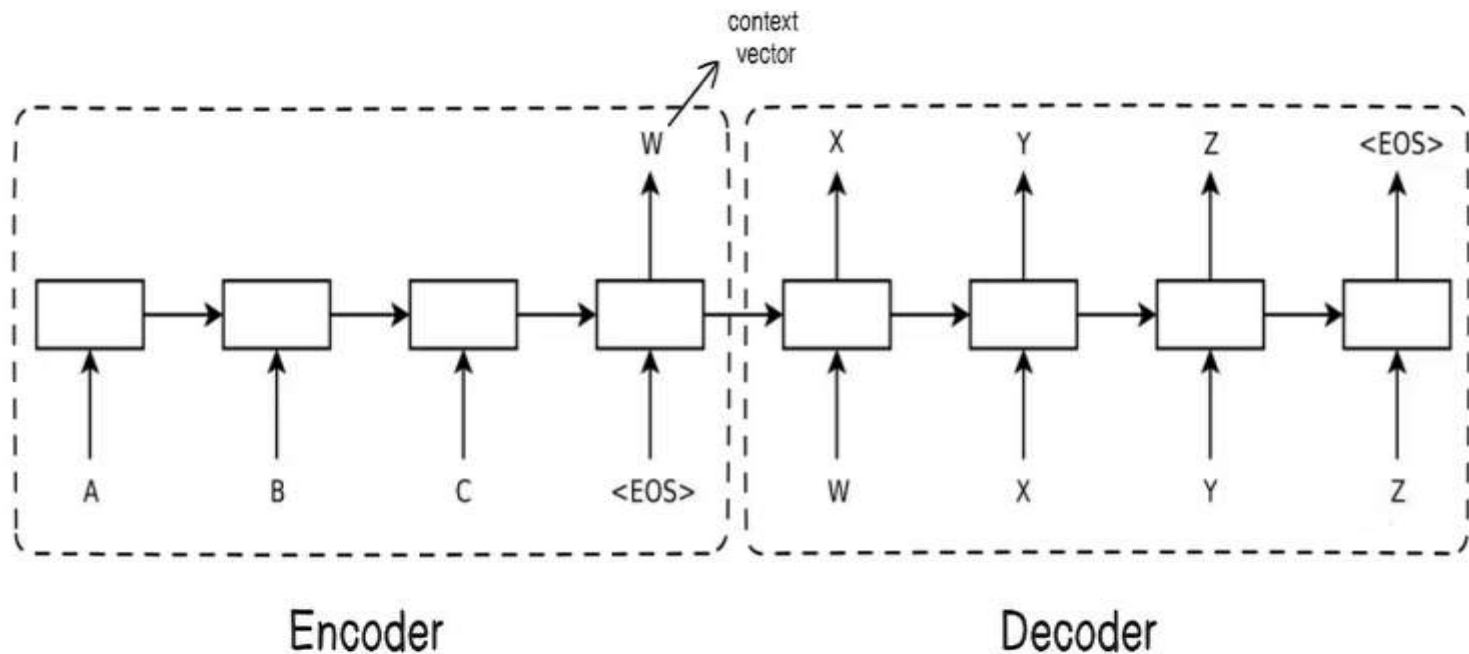
A Seq2Seq model generally contains:

Encoder – Converts input sequence into a context vector.

Decoder – Generates output sequence from the context vector.

Advance Sequence to Sequence Models

Basic Seq2seq Architecture



Advance Sequence to Sequence Models

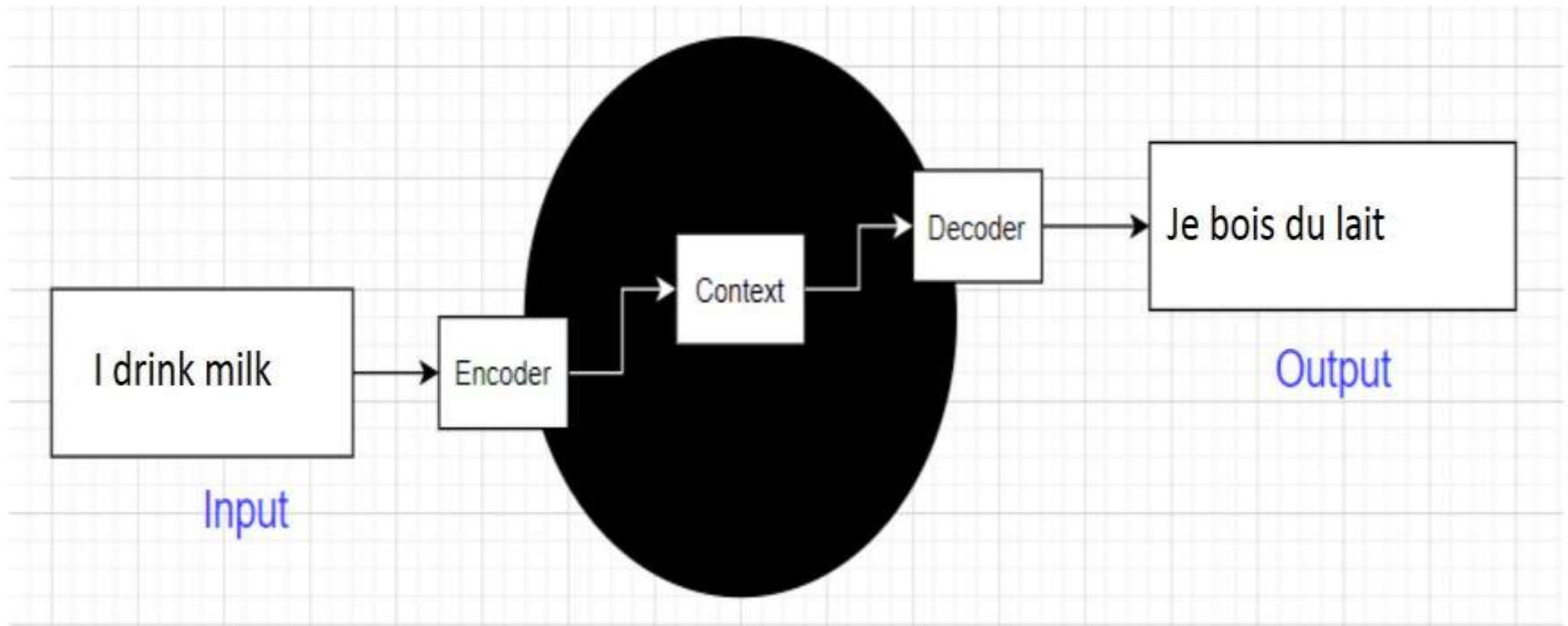


Fig 1. A general architecture of Seq2Seq model

Advance Sequence to Sequence Models

Advantages of Advanced Seq2Seq Models

- Better contextual learning
- Improved long-sequence handling
- Higher translation accuracy
- Faster training using transformers
- Scalable for large datasets

Limitations

- High computational cost
- Requires large training datasets
- Complex tuning process
- Memory intensive

Advance Sequence to Sequence Models

- Advanced Sequence-to-Sequence models revolutionized NLP by improving translation, summarization, and conversational AI systems. Attention mechanisms and Transformer architectures significantly enhanced performance over traditional RNN-based models. Today, most modern NLP systems, including large language models, are based on advanced Seq2Seq concepts.

References

1. <https://towardsdatascience.com/illustrated-guide-to-lstms-and-gru-s-a-step-by-step-explanation-44e9eb85bf21>
<https://www.analyticsvidhya.com/blog/2020/08/a-simple-introduction-to-sequence-to-sequence-models/>

Parul[®]
University

NAAC
GRADE **A++**



<https://paruluniversity.ac.in/>

